WEEK 01 · 1주차

수업 오리엔테이션과 과목 개요

AI기반 빅데이터 프로젝트 · 2026-2

승인된 수강생의 학습을 위한 자료입니다. 퀴즈 정답과 해설은 교수자의 별도 공개 전까지 포함하지 않습니다.

1장. 수업 오리엔테이션과 과목 개요

1. 학습목표

1. (신기술융합) 빅데이터 처리·DevOps·생성형 AI가 하나의 데이터 파이프라인에서 결합되는 구조를 설명할 수 있다.
2. (데이터분석활용) 수집·저장·처리·분석·배포 5계층과 각 계층의 기술 스택을 주차별 학습 순서에 대응시켜 진술할 수 있다.
3. (오픈소스활용) 평가 항목 6종과 과제 4종의 제출 시점·산출물 요건을 파악하고 팀 저장소를 초기화할 수 있다.
4. (신기술융합) 실습 환경의 최소 요구 버전을 점검 스크립트로 확인하고 미충족 항목을 식별할 수 있다.
5. (오픈소스활용) 생성형 AI 산출물에 대한 검증 의무와 활용 이력 기록 규범을 진술할 수 있다.

---

2. 내용

2.1 왜 빅데이터·DevOps·생성형 AI를 한 과목에서 다루는가

데이터 처리 기술을 단일 주제로 학습하면 개별 도구의 사용법은 익히지만 도구 사이의 접합부를 다루지 못한다. 실제 현장에서 장애가 발생하는 지점은 대부분 접합부다. 수집기가 만든 파일 포맷을 처리 엔진이 읽지 못하거나, 로컬에서 동작하던 분석 코드가 클러스터에서 실패하거나, 모델은 학습되었으나 서비스로 배포되지 못하는 상황이 여기에 해당한다. 본 교과목이 세 축을 한 학기에 묶는 이유는 이 접합부를 학습 대상으로 삼기 위함이다.

세 축의 역할은 다음과 같이 구분된다. 빅데이터 처리는 단일 노드로 감당되지 않는 규모의 데이터를 분산 저장·연산하는 방법을 다룬다. DevOps는 그렇게 만든 처리 로직을 재현 가능한 형태로 포장하고 자동으로 배포하는 방법을 다룬다. 생성형 AI는 앞의 두 축을 수행하는 과정에서 코드·설정 초안을 만들고 분석 결과를 문서화하는 보조 수단이며, 동시에 검증 대상이다. 세 축은 병렬로 나열되는 것이 아니라 하나의 파이프라인을 서로 다른 층위에서 지지한다.

학기 운영은 이 구조를 반복 교차하도록 배치되어 있다. 전체 16주 중 빅데이터 처리에 6주(3~5주, 9~11주), DevOps에 3주(6·8·12주), 생성형 AI·머신러닝에 3주(2·13·14주), 오픈소스 프로젝트에 3주(1·15·16주)를 배정하고, 7주차 1주를 중간 정리와 중간고사에 사용한다. 각 트랙은 앞선 트랙의 산출물을 입력으로 받는다. 예를 들어 4주차에 적재한 데이터는 5주차 맵리듀스(MapReduce)의 입력이 되고, 그 처리 결과는 11주차 아파치 스파크(Apache Spark)와의 성능 비교 대상이 되며, 6주차와 12주차에서 만든 컨테이너·자동화 구성 위에서 실행된다.

그림 1-1. 16주 로드맵 — 4개 트랙(빅데이터 처리 6주, DevOps 3주, 생성형 AI·ML 3주, 오픈소스 프로젝트 3주)과 정리·시험 1주가 색으로 구분된 주차 노드로 배치된다. 전반기 1~8주는 저장과 처리의 기반을 세우고, 후반기 9~16주는 그 기반 위에 스트리밍·분석·배포를 누적한다. 16주차는 오픈소스 트랙에 속하지만 기말고사와 프로젝트 발표가 함께 배치된다. (주차 노드를 클릭하면 학습 범위가 표시되는 인터랙티브 도식은 03_도식_HTML/CH01_수업오리엔테이션과과목개요.html 참조)



그림 1-1. 16주 로드맵

〔프로젝트 연결〕 1주차에 개설하는 팀 저장소는 학기 말에 폐기되는 실습용 저장소가 아니다. 5주차 과제1의 맵리듀스 코드, 11주차 과제2의 성능 측정표, 13주차 과제3의 검증 리포트가 모두 같은 저장소에 누적되고, 16주차에는 그 저장소의 커밋 이력 자체가 프로젝트 평가 근거가 된다. 저장소 구조를 1주차에 정확히 잡아 두는 것이 학기 전체의 작업량을 줄인다.

2.2 데이터 파이프라인 전 과정 조망

본 교과목이 다루는 모든 기술은 데이터 파이프라인의 다섯 계층 중 하나에 위치한다. 계층은 수집(Ingest), 저장(Store), 처리(Process), 분석(Analyze), 배포(Serve)로 구분된다. 계층을 먼저 고정해 두면 새로운 도구를 접했을 때 "이 도구가 어느 계층의 어떤 문제를 푸는가"를 기준으로 위치를 판단할 수 있고, 도구 사이의 대체 관계와 보완 관계를 혼동하지 않게 된다.

〔개념〕 데이터 파이프라인(data pipeline) 은 원천 데이터가 소비 가능한 형태로 변환되어 사용자 또는 후속 시스템에 도달하기까지의 처리 경로를 뜻한다. 계층 구분은 물리적 구성 요소의 구분이 아니라 책임의 구분이다. 하나의 도구가 두 계층에 걸치는 경우도 있다. 예를 들어 Elasticsearch는 색인 저장소이면서 집계 분석 엔진이므로 저장과 분석 양쪽에 걸친다.

그림 1-2. 데이터 파이프라인 5계층과 기술 스택 — 다섯 계층이 좌에서 우로 연결되며 각 계층의 기술 스택과 집중 학습 주차가 표기된다. 계층별 구성은 아래와 같다.

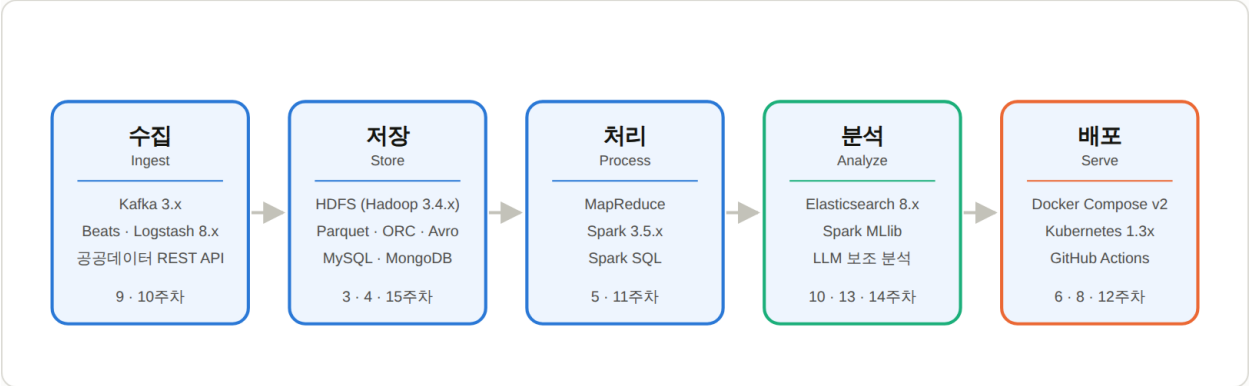


그림 1-2. 데이터 파이프라인 5계층과 기술 스택

계층	책임	기술 스택	집중 주차
수집 (Ingest)	원천에서 데이터를 끌어와 파이프라인에 투입	Kafka 3.x, Beats·Logstash 8.x, 공공데이터 REST API	9·10주차
저장 (Store)	재처리 가능한 형태로 원본과 사본을 보관	HDFS(Hadoop 3.4.x), Parquet·ORC·Avro, MySQL·MongoDB	3·4·15주차
처리 (Process)	정제·집계·결합 연산 수행	MapReduce, Spark 3.5.x, Spark SQL	5·11주차
분석 (Analyze)	검색·집계·예측으로 의미 도출	Elasticsearch 8.x, Spark MLlib, LLM 보조 분석	10·13·14주차
배포 (Serve)	재현 가능한 형태로 포장하여 운영 환경에 반영	Docker Compose v2, Kubernetes 1.3x, GitHub Actions	6·8·12주차

계층 구분이 실제로 유용한 지점은 기술 선택 국면이다. 수집 계층에서 Kafka를 도입할지 여부는 데이터의 도착 간격과 소비자 수로 결정되며, 저장 계층에서 Parquet을 선택할지 여부는 조회 패턴이 열 단위 집계인지 행 단위 조회인지로 결정된다. 도구의 인기도가 아니라 계층별 요구사항이 선택 근거가 된다는 점을 학기 내내 반복적으로 확인한다.

〔더 알아보기〕 각 계층의 공식 문서를 학기 시작 시점에 즐겨찾기에 등록해 둔다. Hadoop(hadoop.apache.org/docs/stable), Spark(spark.apache.org/docs/latest), Kafka(kafka.apache.org/documentation), Elastic(elastic.co/guide), Kubernetes(kubernetes.io/docs), Docker Compose(docs.docker.com/compose). 생성형 AI 산출물을 검증할 때 기준이 되는 것은 검색 결과가 아니라 사용 중인 버전의 공식 문서다.

2.3 평가 체계와 과제 4종의 연결 구조

평가는 여섯 항목으로 구성되며 합계 100%다. 팀 프로젝트가 30%로 가장 크고, 중간고사와 기말고사가 각 20%, 과제·출석·참여도가 각 10%다. 시험 40%와 상시 평가 30%가 개인 성취를, 프로젝트 30%가 팀 성취를 측정한다. 이 구조는 개인 학습을 소홀히 하고 팀에 의존하는 방식으로도, 팀 기여 없이 시험 점수만으로도 상위 성취에 도달하기 어렵도록 설계되어 있다.

그림 1-3. 평가 항목별 비중 (전체 100% 기준, 단위 %) — 가로 막대 차트로 표현하며 각 막대에 값을 직접 표기한다. 팀 프로젝트 30, 중간고사 20, 기말고사 20, 과제 10, 출석 10, 참여도 10이다. 단일 항목으로는 프로젝트가 가장 크지만 시험 두 항목의 합 40%가 그보다 크다. 출처는 2026학년도 2학기 강의계획서의 평가 방법 항목이다.

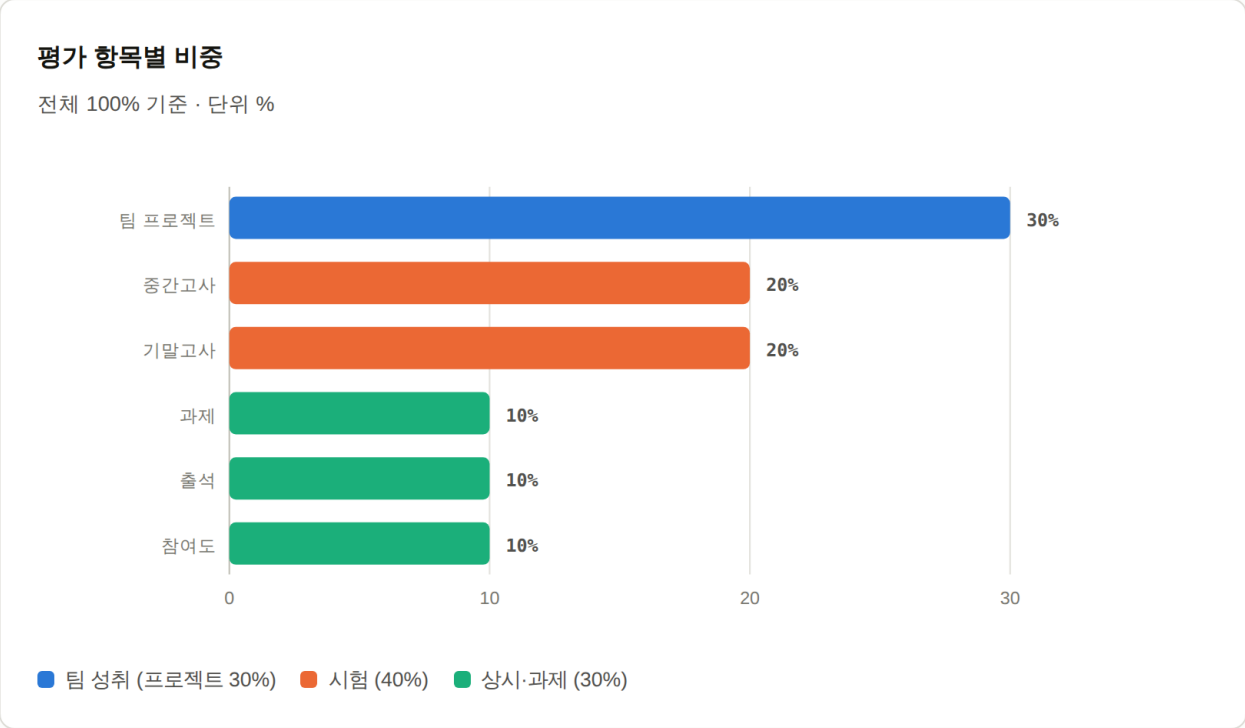


그림 1-3. 평가 항목별 비중 (전체 100% 기준, 단위 %)

표 1-1. 16주 주차별 학습내용·트랙·교재·주요 활동

주차	기간	학습 내용	트랙	교재	주요 활동·평가
1	09.01~09.05	수업 오리엔테이션과 과목 개요	오픈소스	주교재	환경 점검, 팀 저장소 개설
2	09.08~09.12	빅데이터 개론과 생성형 AI 활용	생성형 AI·ML	주교재, 부교재①	LLM 설계 초안 결함 탐지
3	09.15~09.19	하둡 파일시스템 설치와 리눅스 실습환경	빅데이터	주교재	Docker Compose 클러스터 기동
4	09.22~09.26	하둡 파일시스템 실습	빅데이터	주교재	공공데이터 적재, 포맷 비교
5	09.29~10.03	맵리듀스 프로그래밍	빅데이터	주교재	WordCount·역색인 구현
6	10.06~10.10	컨테이너와 DevOps 기초	DevOps	부교재③	과제1 제출, 팀·주제 확정
7	10.13~10.17	중간 정리와 중간고사 대비	정리·시험	—	중간고사 20%
8	10.20~10.24	Kubernetes와 오케스트레이션	DevOps	부교재③	워크로드 배포, HPA 관측

주차	기간	학습 내용	트랙	교재	주요 활동·평가
9	10.27~10.31	카프카를 이용한 데이터 송수신	빅데이터	주교재	Producer·Consumer 구현
10	11.03~11.07	엘라스틱 스택: 수집·검색·시각화	빅데이터	주교재	로그 수집, 대시보드 구성
11	11.10~11.14	아파치 스파크	빅데이터	주교재	과제2 제출, 중간 발표
12	11.17~11.21	CI/CD 파이프라인 구축	DevOps	부교재③	GitHub Actions 자동화
13	11.24~11.28	생성형 AI 기반 데이터 분석	생성형 AI·ML	부교재①	과제3 제출
14	12.01~12.05	머신러닝과 빅데이터 분석	생성형 AI·ML	부교재②	MLlib 파이프라인 구성
15	12.08~12.12	데이터 연계와 프로젝트 개발	오픈소스	주교재	전체 파이프라인 통합 배포
16	12.15~12.19	프로젝트 최종 발표와 소논문	오픈소스	—	과제4·기말고사 20%·발표 30%

과제 4종은 독립된 네 개의 숙제가 아니라 하나의 프로젝트를 네 시점에서 절단한 단면이다. 과제1이 처리 계층의 기본기를, 과제2가 처리 엔진 간 성능 비교를, 과제3이 분석 계층의 생성형 AI 활용과 검증을, 과제4가 전체 결과의 문서화와 공개를 담당한다. 앞 과제의 산출물을 뒤 과제가 입력으로 사용하므로 지연 제출은 후속 과제의 작업량을 누적적으로 증가시킨다.

표 1-2. 과제 4종 제출 캘린더와 산출물 요건

과제	주제	제출 주차	산출물 요건	후속 연결
과제1	맵리듀스 프로그래밍	6주차 (10.06~10.10)	저장소 소스, 실행 로그, 입출력 샘플	과제2의 성능 비교 기준선
과제2	Hadoop·Spark 처리 비교	11주차 (11.10~11.14)	저장소 소스, 측정 조건 명시 성능 표	프로젝트 중간 발표 자료
과제3	생성형 AI 활용 데이터 분석·문서화	13주차 (11.24~11.28)	분석 코드, 검증 리포트, AI 활용 이력	소논문의 방법·실험 절
과제4	팀 프로젝트 소논문·발표	16주차 (12.15~12.19)	소논문, 공개 저장소, 발표자료	프로젝트 30% 평가 근거

〔개념〕 전공능력(Program Outcome, PO) 은 학과가 졸업 시점에 학생이 갖추어야 할 능력으로 정의한 항목이다. 본 교과목은 데이터분석활용능력(PO4), 오픈소스활용능력(PO5), 신기술융합능력(PO6) 세 항목을 담당한다. 각 평가 항목은 특정 전공능력의 성취 근거로 사용되므로 평가 항목과 전공능력의 대응 관계를 학기 초에 확인해 두는 것이 유익하다.

표 1-3. 전공능력 반영률과 평가 항목 대응 (교안 설계 기준, 합계 100%)

전공능력	배정 주차	주차 수	대응 평가 항목	반영률
PO4 데이터분석활용능력	3~5, 9~11주	6	중간고사 20% + 과제 10% + 프로젝트 처리·분석 10%	40%
PO6 신기술융합능력	2, 6, 8, 12~14주	6	기말고사 20% + 프로젝트 배포 구성 10%	30%
PO5 오픈소스활용능력	1, 15, 16주	3	프로젝트 저장소·소논문 10% + 출석 10% + 참여도 10%	30%
통합 정리	7주	1	중간고사에 포함	—

2.4 생성형 AI 활용 규범과 검증 의무

본 교과목은 생성형 AI 사용을 금지하지 않는다. 다만 사용 조건을 명시한다. 첫째, 생성된 코드·설정·문장은 검증 없이 제출물에 포함하지 않는다. 둘째, 활용한 사실과 범위를 저장소에 기록한다. 셋째, 검증하지 않은 산출물로 인한 오류의 책임은 제출자에게 있다. 이 세 조건은 학문적 정직성 요구를 실습 절차로 환산한 것이며, 과제3과 프로젝트 평가에서 실제 점검 대상이 된다.

검증 의무를 실행 가능한 절차로 만들면 네 단계가 된다. 질의를 설계할 때 사용 중인 버전과 제약을 명시하고, 생성된 산출물을 실행 환경에서 직접 돌려 보며, 해당 버전의 공식 문서와 대조하고, 활용 범위와 수정 내역을 기록한다. 검증에 실패하면 결과를 폐기하고 질의를 다시 설계한다.

그림 1-4. 생성형 AI 산출물 검증 절차 — ① 질의 설계(역할·제약·버전 명시) → ② 산출물 생성(코드·설정 초안 확보) → ③ 실행 검증(실행 결과·공식 문서 대조) → ④ 기록·표기(AI_USAGE.yaml 갱신)로 진행되며, ③에서 검증에 실패하면 결과를 폐기하고 ①로 되돌아가는 순환 구조를 이룬다. 검증은 마지막에 한 번 수행하는 확인 작업이 아니라 질의 설계 단계부터 시작된다.

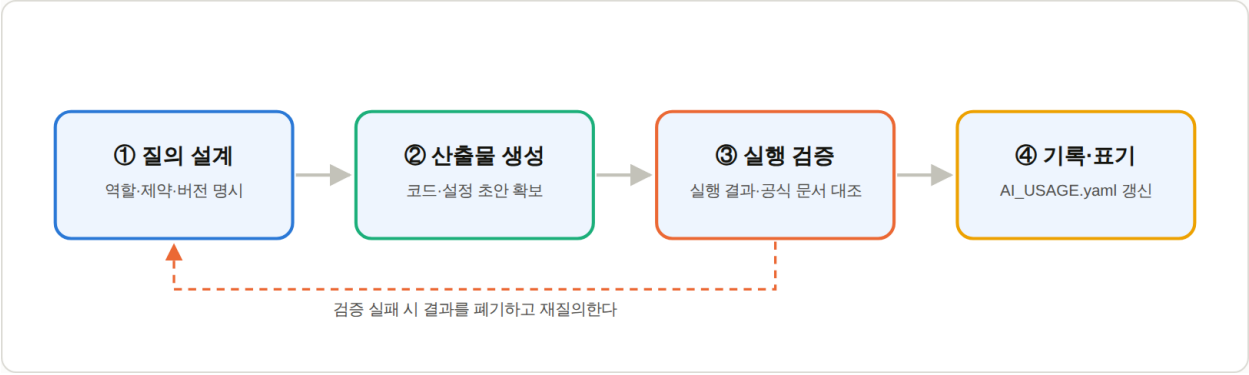


그림 1-4. 생성형 AI 산출물 검증 절차

〔AI 함정〕 생성형 AI는 학습 시점 이후에 변경된 명령 체계를 반영하지 못한다. 대표적으로 docker-compose(하이픈 형태, Compose v1)와 docker compose(공백 형태, Compose v2)를 혼용해 제시하는 사례, Hadoop 2.x 기준의 mapred 설정 키를 3.4.x 환경에 제시하는 사례, 존재하지 않는 명령 옵션을 그럴듯한 이름으로 만들어 내는 사례가 반복된다. 판별 방법은 단순하다. 제시된 명령에 --help를 붙여 해당 옵션이 실제 출력에 나타나는지 확인하면 된다. 출력에 없으면 그 옵션은 현재 버전에 존재하지 않는다.

3. 실습

실습은 학기 전체의 기준선을 만드는 작업이다. 결과물은 점검 스크립트 실행 로그, 초기화된 팀 저장소, 생성형 AI 활용 이력 파일 세 가지다.

표 1-4. 실습 환경 최소 요구 사항

구성 요소	요구 버전	확인 명령	비고
Docker Engine	24.0 이상	docker --version	3주차부터 상시 사용
Docker Compose	v2.20 이상	docker compose version	공백 형태 명령을 사용한다
Git	2.40 이상	git --version	git switch 사용
Python	3.10~3.11	python3 --version	PySpark 3.5.x 권장 범위
JDK	11 (LTS)	java -version	Hadoop 3.4.x·Spark 3.5.x 공통 지원
메모리 / 디스크	8GB 이상 / 20GB 이상	—	클러스터 실습 기준 16GB 권장

3.1 개발 환경 점검 스크립트 실행

다섯 구성 요소의 설치 여부와 버전을 한 번에 확인한다. 셸을 사용할 수 없는 환경에서는 동등한 파이썬 스크립트를 사용한다. 출력에 MISS가 있으면 해당 항목을 설치한 뒤 재실행한다.

```
#!/usr/bin/env bash
# check_env.sh — 실습 환경 점검
set -u

check() {
  local name="$1"; shift
  if command -v "$1" >/dev/null 2>&1; then
    printf '%-10s OK   %s\n' "$name" "$("$@" 2>&1 | head -n 1)"
  else
    printf '%-10s MISS 설치 필요\n' "$name"
  fi
}

check Docker  docker --version
check Compose docker compose version
check Git     git --version
check Python  python3 --version
check Java    java -version
```

```
import shutil
import subprocess
import sys

TOOLS = {
    "Docker":  ["docker", "--version"],
    "Compose": ["docker", "compose", "version"],
    "Git":     ["git", "--version"],
    "Python":  [sys.executable, "--version"],
    "Java":    ["java", "-version"],
}

def probe(cmd):
    if shutil.which(cmd[0]) is None:
        return None
    done = subprocess.run(cmd, capture_output=True, text=True)
```

```
    text = done.stdout or done.stderr
    return text.splitlines()[0].strip() if text else ""

for name, cmd in TOOLS.items():
    line = probe(cmd)
    state = "OK  " if line is not None else "MISS"
    print(f"{name:10} {state} {line or '설치 필요'}")
```

코드 1-1. 개발 환경 점검 스크립트(Bash·Python 탭 구성). command -v로 실행 파일 존재를 먼저 확인하므로 미설치 항목에서 중단되지 않는다. java -version은 표준 오류로 출력하므로 2>&1 병합이 필요하다.

3.2 Docker Compose 동작 확인

버전 문자열만으로는 컨테이너가 실제로 기동되는지 알 수 없다. 두 개의 경량 이미지로 스모크 테스트를 수행한다. 두 서비스가 각각 버전을 출력하고 종료하면 3주차 하둡 클러스터 실습을 진행할 수 있는 상태다.

```
# compose.yaml — 실습 환경 스모크 테스트
services:
  python-probe:
    image: python:3.11-slim
    command: ["python", "-c", "import sys; print('python', sys.version.split()[0])"]
  jdk-probe:
    image: eclipse-temurin:11-jre
    command: ["java", "-version"]

docker compose -f compose.yaml up --abort-on-container-exit
docker compose -f compose.yaml down --remove-orphans
docker image ls --filter reference='python:3.11-slim'
```

코드 1-2. Docker Compose 스모크 테스트. Compose v2에서는 최상위 version 키가 폐기되었으므로 작성하지 않는다.

3.3 GitHub 팀 저장소 초기화

학기 전체에서 사용할 디렉터리 구조를 처음에 확정한다. 파이프라인 5계층에 대응하는 src/ingest, src/process, src/serve 구조를 사용하면 이후 과제의 배치 위치를 매번 논의하지 않아도 된다. 원본 데이터와 중간 산출물은 저장소에 포함하지 않는다.

```
mkdir -p bdp-team07 && cd bdp-team07
git init -b main
mkdir -p docs data/raw notebooks src/ingest src/process src/serve infra .github/workflows
printf '# bdp-team07\n\n국립군산대 AI기반빅데이터프로젝트 팀 저장소\n' > README.md
git add -A
git commit -m "chore: 팀 저장소 초기 구조 생성"

# GitHub CLI로 원격 저장소를 생성하고 연결한다
gh auth status
gh repo create bdp-team07 --private --source=. --remote=origin --push

# .gitignore — 데이터·비밀값·빌드 산출물 제외
data/raw/
data/interim/
*.parquet
*.orc
*.jar
.env
.venv/
__pycache__/
.ipynb_checkpoints/
target/
```

코드 1-3. 팀 저장소 초기화. git init -b main으로 기본 브랜치 이름을 명시하여 로컬 설정에 따른 차이를 없앤다.

3.4 브랜치 전략과 커밋 규칙 적용

기능 단위 브랜치를 만들고 풀 리퀘스트로 통합하는 방식을 1주차부터 적용한다. 커밋 메시지는 타입(범위): 요약 형식을 사용한다. 커밋 이력은 참여도 평가와 프로젝트 기여도 판단의 근거가 되므로 작업 단위를 작게 유지한다.

```
git switch -c feat/ingest-open-api
git add src/ingest/collect_open_api.py
git commit -m "feat(ingest): 공공데이터 API 수집기 초안 추가"
git push -u origin feat/ingest-open-api

gh pr create --base main --title "feat(ingest): 공공데이터 수집기" --body "검증 로그 첨부"
gh pr list --state open
```

코드 1-4. 기능 브랜치와 풀 리퀘스트 생성. git switch -c는 Git 2.23 이후 도입된 브랜치 전환 명령이다.

3.5 생성형 AI 도구 계정 준비와 사용량 관리

사용할 도구를 팀 단위로 하나 이상 지정하고 무료 사용 한도와 갱신 주기를 확인한다. 한도가 소진되면 과제 마감 직전에 작업이 중단되므로, 한도가 큰 작업(대량 코드 생성)과 작은 작업(문장 다듬기)을 구분하여 배분한다. API 키는 저장소에 올리지 않고 .env로 분리하며 .gitignore에 포함되어 있는지 확인한다.

```
# 비밀값을 저장소 밖으로 분리한다
printf 'LLM_API_KEY=여기에_키를_입력한다\n' > .env
git check-ignore -v .env          # .gitignore에 걸리는지 확인
git log --oneline -S 'API_KEY'    # 과거 커밋에 키가 섞였는지 점검

import csv
import datetime
import pathlib

LOG = pathlib.Path("docs/llm_usage_log.csv")
LOG.parent.mkdir(parents=True, exist_ok=True)
is_new = not LOG.exists()

with LOG.open("a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    if is_new:
        writer.writerow(["date", "tool", "task", "requests", "verified"])
    writer.writerow([
        datetime.date.today().isoformat(),
        "generative-ai-tool",
        "check_env.sh 초안 생성",
        3,
        "yes",
    ])
print("기록 완료:", LOG)
```

코드 1-5. 비밀값 분리와 사용량 기록. git check-ignore -v는 해당 경로를 제외시키는 규칙의 파일과 행 번호를 함께 출력한다.

3.6 AI 활용 이력 작성과 검증 실행

생성형 AI로 만든 산출물을 저장소에 넣기 전에 활용 이력을 남기고 검증을 수행한다. 검증은 제시된 명령과 옵션이 현재 설치된 버전에 실제로 존재하는지 확인하는 것에서 시작한다.

```
# AI_USAGE.yaml — 생성형 AI 활용 이력 (제출 전 갱신)
- date: "2026-09-03"
  tool: "생성형 AI 대화 도구"
  purpose: "환경 점검 스크립트 초안 생성"
  prompt_summary: "bash 5.x 기준으로 docker/git/python/java 버전을 한 줄씩 출력"
```

```
  output_used: "src/tools/check_env.sh 1~20행"
  verification: "Ubuntu 22.04에서 실행, java 미설치 시 MISS 출력 확인"
  modification: "command -v 존재 확인 추가, 표준 오류 병합 처리 수정"
  verified_by: "학번 20220000 홍길동"

# 제시된 옵션이 현재 버전에 실제로 존재하는지 확인한다
docker compose up --help | grep -- '--abort-on-container-exit'
git switch --help | head -n 5
python3 -c "import platform; print(platform.python_version())"

# 확인 결과를 이력과 함께 커밋한다
git add AI_USAGE.yaml src/tools/check_env.sh
git commit -m "docs(ai): 환경 점검 스크립트 AI 활용 이력 기록"
```

코드 1-6. AI 활용 이력과 검증 절차. grep이 결과를 출력하지 않으면 해당 옵션은 현재 버전에 존재하지 않는 것이므로 산출물을 폐기하고 재질의한다.

〔AI 함정〕 생성형 AI에 설치 명령을 요청하면 특정 버전 번호를 확정적으로 제시하는 경우가 많다. 그러나 배포판·아키텍처에 따라 제공되는 패키지 버전이 다르므로 제시된 번호가 실제 설치 가능한 버전과 일치하지 않을 수 있다. 버전은 생성된 답변이 아니라 docker --version과 같이 설치된 환경의 출력으로 확정한다. 표 1-4의 요구 버전을 충족하는지가 판단 기준이며, 그보다 높은 마이너 버전은 허용된다.

〔프로젝트 연결〕 이 실습에서 만든 AI_USAGE.yaml은 13주차 과제3과 16주차 소논문 제출 시 필수 첨부 항목이다. 학기 말에 소급해서 작성하면 어떤 산출물의 어느 부분이 생성형 AI로 만들어졌는지 복원할 수 없다. 커밋 단위로 즉시 갱신하는 습관을 1주차부터 유지한다.

---

4. 요약

- 3축 융합 — 빅데이터 처리는 규모를, DevOps는 재현성과 배포를, 생성형 AI는 작성 보조와 검증 대상을 담당한다. 세 축은 병렬 주제가 아니라 하나의 파이프라인을 지지하는 서로 다른 층위다.
- 파이프라인 5계층 — 수집·저장·처리·분석·배포로 구분한다. 새 도구를 접했을 때 어느 계층의 어떤 문제를 푸는지로 위치를 판단하면 대체 관계와 보완 관계를 혼동하지 않는다.
- 16주 배치 — 전체 16주 중 빅데이터 6주, DevOps 3주, 생성형 AI·ML 3주, 오픈소스 3주, 정리·시험 1주로 배정된다. 후반기 내용은 전반기 산출물을 입력으로 사용한다.
- 평가 100% 구성 — 프로젝트 30%, 중간·기말 각 20%, 출석·과제·참여도 각 10%다. 단일 항목은 프로젝트가 가장 크지만 시험 두 항목의 합 40%가 그보다 크다.
- 과제 4종의 연속성 — 6주차 과제1, 11주차 과제2, 13주차 과제3, 16주차 과제4는 하나의 프로젝트를 네 시점에서 절단한 단면이며 앞 과제의 산출물이 뒤 과제의 입력이 된다.
- 실습 환경 기준선 — Docker 24 이상, Compose v2.20 이상, Git 2.40 이상, Python 3.10~3.11, JDK 11을 기준으로 한다. JDK 11은 Hadoop 3.4.x와 Spark 3.5.x가 공통으로 지원하는 LTS 버전이다.
- AI 검증 4단계 — 질의 설계, 산출물 생성, 실행 검증, 기록·표기로 구성된다. 검증에 실패하면 결과를 폐기하고 재질의하며, 활용 이력은 커밋 단위로 즉시 남긴다.
- 다음 주차 예고 — 2주차에는 빅데이터 5V와 스케일아웃이 필요한 규모의 임계점을 정량적으로 다루고, 생성형 AI로 만든 파이프라인 설계 초안에서 결함을 찾는 실습을 수행한다.

---

5. 퀴즈

1. 생성형 AI가 제안한 명령을 채택하기 전 가장 적절한 행동은?

1) 길이 확인

2) --help와 실제 실행으로 검증

3) 경고 삭제

4) 버전 생략

정답: 2 해설: 현재 환경에서 옵션과 결과가 실제로 존재하는지 확인해야 한다.

2. 재현 가능한 환경 보고서에 포함할 항목은?

1) 개인 비밀번호

2) 결과 이미지 하나

3) OS·Python·Java·Docker 버전

4) 팀원 학번

정답: 3 해설: 실행 환경을 재구성할 수 있는 관찰값이 필요하다.

3. 데이터 파이프라인 계층을 나누는 가장 좋은 기준은?

1) 로고 색

2) 도구 가격

3) 인기 순위

4) 입력·출력·실패 책임

정답: 4 해설: 하나의 도구가 여러 계층에 걸칠 수 있어 책임 경계가 중요하다.

4. AI_USAGE.yaml의 목적은?

1) AI 사용과 검증 결정을 추적

2) API 키 공유

3) 성적 자동 확정

4) 오류 은폐

정답: 1 해설: 프롬프트와 검증 근거를 남겨 학문적 정직성과 재현성을 확보한다.

통제어 참조

본문에서 정의한 기술 용어의 영구 식별자와 최초 정의 위치입니다. 정의는 본문 한 곳에서만 유지합니다.

통제어	영문·식별 URI	최초 정의
데이터 파이프라인 (data pipeline)	https://knowledgemap.kr/id/concept/ai-bigdata-project/datapipeline	이 주차
전공능력 (Program Outcome, PO)	https://knowledgemap.kr/id/concept/ai-bigdata-project/programoutcomepo	이 주차